# Exception Handling – Demonstration of 10 Exception Classes

**Observation Task:** Task-7 **Date:** 09/10/2026

## ■ Objective / Aim

To design and implement a Java program that demonstrates ten different exception classes – both checked and unchecked – using try, catch, and finally blocks, and to identify each exception's type and the reason it occurs.

## ■ System Requirements

**Hardware**

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

**Software**

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

## ■ Algorithm / Logic Description

1. Create a class named **ExceptionDemo**.
2. Define a common helper method show() that prints the exception number, name, the code that triggered it, the caught exception's class, whether it is checked or unchecked, and the reason for the failure.
3. Write a separate method for each exception scenario, wrapping the risky code in a try block and catching the specific exception type:
   - ArithmeticException – division by zero.
   - ArrayIndexOutOfBoundsException – accessing an array index beyond its bounds.
   - NullPointerException – calling a method on a null reference.
   - NumberFormatException – parsing a non-numeric string as a number.
   - ClassCastException – casting an object to an incompatible type.
   - StringIndexOutOfBoundsException – accessing a character position beyond a string's length.
   - NegativeArraySizeException – creating an array with a negative size.
   - ArrayStoreException – storing an incompatible type in an array.
   - FileNotFoundException (checked) – opening a file that does not exist.
   - ClassNotFoundException (checked) – loading a class that cannot be found.
4. Add a finally block to the ArithmeticException demo to show that it always executes, regardless of whether an exception occurred.
5. Determine checked vs. unchecked by checking instanceof RuntimeException – unchecked exceptions extend RuntimeException, checked exceptions do not.
6. In main(), call each demo method in sequence and print a closing message once all exceptions have been handled.

## ■ UML Class Diagram




*Relationship: All exception classes used in this program ultimately **inherit** from Throwable via Exception. Classes extending RuntimeException are **unchecked** (not enforced by the compiler), while FileNotFoundException and ClassNotFoundException are **checked** (the compiler forces them to be caught or declared). ExceptionDemo's methods each **use** (association) a specific exception type by triggering and catching it, and all route through the shared show() helper to report results uniformly.*

## ■ Source Code

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;

public class ExceptionDemo {

    // Common method to display the details of a caught exception
    static void show(int no, String name, String code, Exception e, String reason) {
        String type = (e instanceof RuntimeException) ? "Unchecked" : "Checked";
        System.out.println("\n" + no + ". " + name);
        System.out.println("   Code    : " + code);
        System.out.println("   Caught  : " + e.getClass().getName());
        System.out.println("   Type    : " + type);
        System.out.println("   Reason  : " + reason);
    }

    // 1. ArithmeticException
    static void arithmeticDemo() {
        try {
            int a = 10, b = 0;
            int result = a / b;
            System.out.println(result);
        } catch (ArithmeticException e) {
            show(1, "ArithmeticException", "int result = 10 / 0;", e,
                    "Integer division by zero");
        } finally {
            System.out.println("   Finally : finally block always executes");
```

```
    }
}

// 2. ArrayIndexOutOfBoundsException
static void arrayIndexDemo() {
    try {
        int[] arr = new int[3];
        arr[5] = 10;
    } catch (ArrayIndexOutOfBoundsException e) {
        show(2, "ArrayIndexOutOfBoundsException", "arr[5] = 10;  (size is 3)", e,
                "Array index is outside the range 0 to 2");
    }
}

// 3. NullPointerException
static void nullPointerDemo() {
    try {
        String text = null;
        int length = text.length();
        System.out.println(length);
    } catch (NullPointerException e) {
        show(3, "NullPointerException", "text.length();  (text = null)", e,
                "Method called on a null reference");
    }
}

// 4. NumberFormatException
static void numberFormatDemo() {
    try {
        int value = Integer.parseInt("abc");
        System.out.println(value);
    } catch (NumberFormatException e) {
        show(4, "NumberFormatException", "Integer.parseInt(\"abc\");", e,
                "String does not contain a valid number");
    }
}

// 5. ClassCastException
static void classCastDemo() {
    try {
        Object obj = "Hello";
        Integer num = (Integer) obj;
        System.out.println(num);
    } catch (ClassCastException e) {
        show(5, "ClassCastException", "(Integer) obj;  (obj holds a String)", e,
                "Object cast to an incompatible type");
    }
}

// 6. StringIndexOutOfBoundsException
static void stringIndexDemo() {
    try {
        String s = "Java";
        char ch = s.charAt(10);
        System.out.println(ch);
    } catch (StringIndexOutOfBoundsException e) {
        show(6, "StringIndexOutOfBoundsException", "\"Java\".charAt(10);", e,
                "Character position is outside the string length");
    }
}

// 7. NegativeArraySizeException
static void negativeArraySizeDemo() {
    try {
        int size = -5;
        int[] arr = new int[size];
        System.out.println(arr.length);
    } catch (NegativeArraySizeException e) {
        show(7, "NegativeArraySizeException", "new int[-5];", e,
                "Array created with a negative size");
    }
}

// 8. ArrayStoreException
static void arrayStoreDemo() {
    try {
        Object[] items = new String[2];
        items[0] = 100;
    } catch (ArrayStoreException e) {
```

```
            show(8, "ArrayStoreException", "items[0] = 100;  (String[] array)", e,
                    "Integer stored in an array of Strings");
        }
    }

    // 9. FileNotFoundException (checked)
    static void fileNotFoundDemo() {
        try {
            FileInputStream fis = new FileInputStream("missing_file.txt");
            fis.close();
        } catch (FileNotFoundException e) {
            show(9, "FileNotFoundException", "new FileInputStream(\"missing_file.txt\");", e,
                    "The specified file does not exist");
        } catch (java.io.IOException e) {
            System.out.println("I/O error");
        }
    }

    // 10. ClassNotFoundException (checked)
    static void classNotFoundDemo() {
        try {
            Class.forName("com.example.DoesNotExist");
        } catch (ClassNotFoundException e) {
            show(10, "ClassNotFoundException", "Class.forName(\"com.example.DoesNotExist\");", e,
                    "The named class could not be found");
        }
    }

    public static void main(String[] args) {
        System.out.println("===== DEMONSTRATION OF 10 EXCEPTION CLASSES =====");

        arithmeticDemo();
        arrayIndexDemo();
        nullPointerDemo();
        numberFormatDemo();
        classCastDemo();
        stringIndexDemo();
        negativeArraySizeDemo();
        arrayStoreDemo();
        fileNotFoundDemo();
        classNotFoundDemo();

        System.out.println("\n===== All exceptions handled. Program ended normally. =====");
    }
}
```

## ■ Compilation & Execution Commands

Save the source code exactly as:

```
ExceptionDemo.java
```

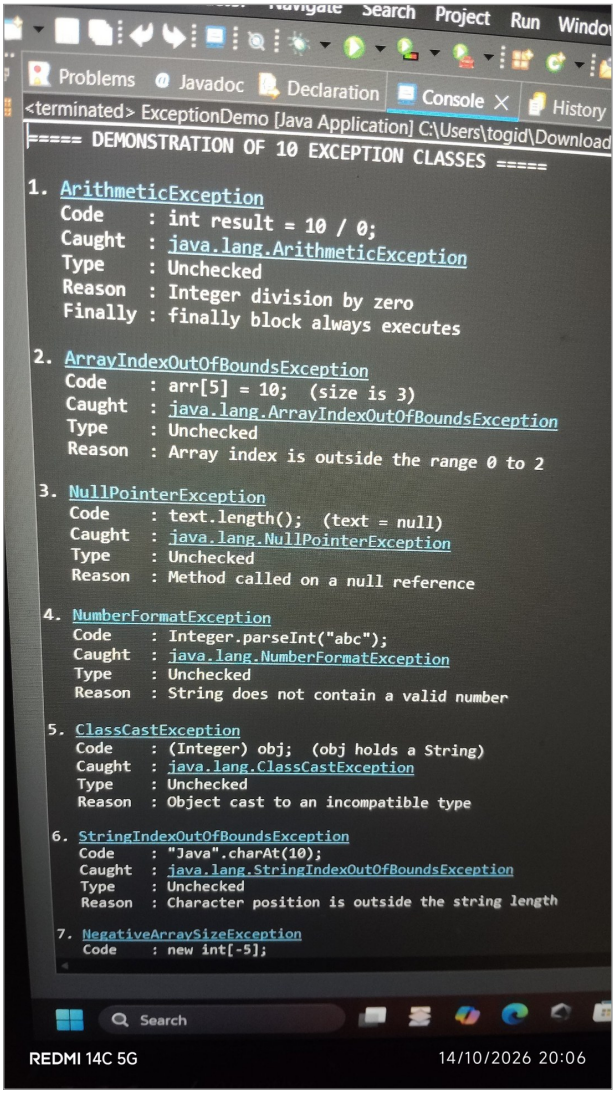
Compile the program:

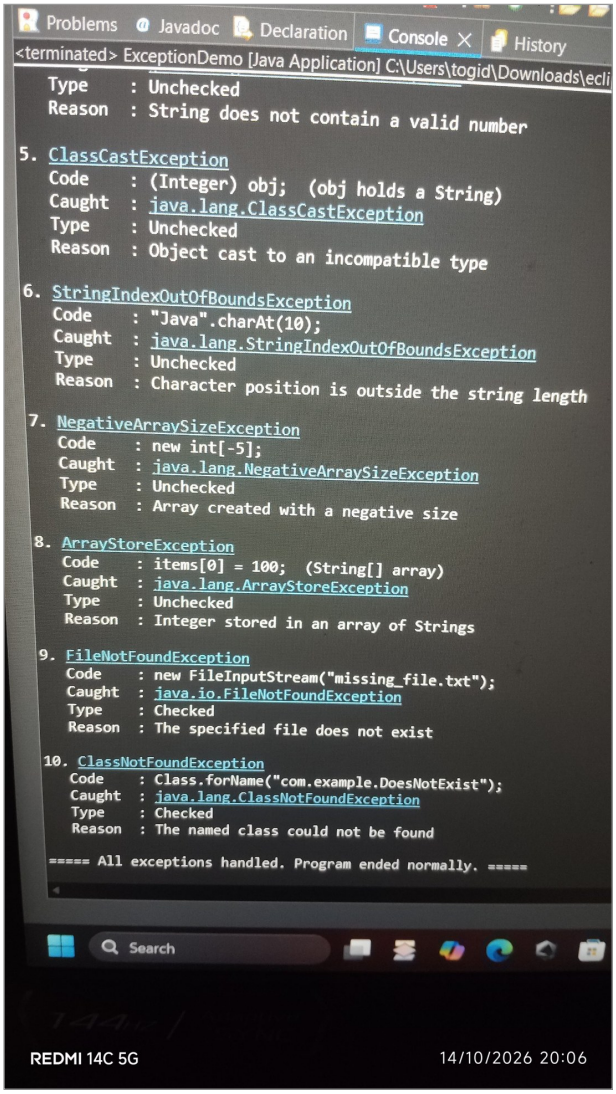
```
javac ExceptionDemo.java
```

Run the compiled program:

```
java ExceptionDemo
```

## ■ Output Screen




## ■ Result

The Exception Handling program was successfully implemented in Java, demonstrating ten distinct exception classes – eight unchecked (RuntimeException subclasses) and two checked (FileNotFoundException, ClassNotFoundException) – each caught using appropriate try-catch blocks, with a finally block shown to execute unconditionally.